

```

P5.l
%{
#include "y.tab.h"
extern int yyval;
}%
%%
[0-9]+ {yyval=atoi(yytext);return num;}
[\+\-\*\V] {return yytext[0];}
[]) {return yytext[0];}
[(] {return yytext[0];}
. {;}
\n {return 0;}
%%

```

```

p5.y
%{
#include<stdio.h>
#include<stdlib.h>
int yyerror();
int yylex();
}%
%token num
%left '+' '-'
%left '*' '/'
%%
input:exp{printf("%d\n",$$);exit(0);}
exp:exp '+' exp{ $$=$1+$3;}
|exp '-' exp{ $$=$1-$3;}
|exp '*' exp{ $$=$1*$3;}
|exp '/' exp {if($3==0){printf("Divide by Zero\n");exit(0);} else $$=$1/$3;}
|('exp'){ $$=$2;}
|num{ $$=$1;};
%%
int yyerror()
{
printf("error");
exit(0);
}
int main()
{
printf("enter expression\n");
yyparse();
}

```

P7: /* Program to recognize a valid variable, which starts with a letter, followed by any number of letters or digits.*/

```

P7.l
%{
#include"y.tab.h"
}%
%%
[a-zA-Z]+ return 1;

```

```

[0-9]+ return d;
[\t] ;
\n return 0;
. return yytext[0];
%%

```

```

P7.y
%{
#include<stdio.h>
int yylex();
int yyerror();
}%
%token l d
%%
var:l s {printf("valid variable");return 0;}
s:s l
|s d
| ;
%%
int main()
{
printf("enter the variable");
yyparse();
}
int yyerror()
{
printf("invalid variable");
exit(0);
}

```

P8: /*Develop, Implement and execute a program using YACC tool to recognize all strings ending with b preceded by n a's using the grammar a n b (note: input n value).*/

```

P8.l
%{
#include "y.tab.h"
}%
%%
a {return A;}
b {return B;}
. {return yytext[0];}
[\n] {return 0;}
%%

```

```

P8.y
%{
#include<stdio.h>
#include<stdlib.h>
int yylex();
int yyerror();
int count=0;
}%
%token A B

```

```

%%
s:A s{count++;}
|B
;
%%
int main()
{
int n;
printf("enter value for n\n");
scanf("%d",&n);
getchar();
printf("enter input string with a's and b's\n");
yyparse();
if(n==count)
printf("valid string\n");
else
yyerror();
return 0;
}
int yyerror()
{
printf("invalid string\n");
exit(0);
}

```

P6.l

```

%{
#include <stdio.h>
#include <stdlib.h>
#include "y.tab.h"
extern int yylval;
}%
%%
[ \t\n]          ; // Ignore whitespace and newline
[+*/=<>-]        { printf("operator is %s\n", yytext); return OP; }
[0-9]+           { yylval = atoi(yytext); printf("number is %d\n", yylval); return DIGIT; }
int[char|bool|float|void|for|do|while|if|else|return { printf("keyword is %s\n", yytext); return KEY; }
[a-zA-Z_][a-zA-Z0-9_]* { printf("identifier is %s\n", yytext); return ID; }
.                ; // Catch-all for any other character
%%
int yywrap() {
    return 1;
}

```

P6.y

```

%{
#include <stdio.h>
#include <stdlib.h>
int id = 0, dig = 0, key = 0, op = 0;
int yyin;
int yylex();
void yyerror();

```

```

%}
%token DIGIT ID KEY OP
%%
input:
    DIGIT input { dig++; }
  | ID input    { id++; }
  | KEY input   { key++; }
  | OP input    { op++; }
  | DIGIT       { dig++; }
  | ID          { id++; }
  | KEY         { key++; }
  | OP          { op++; }
  ;
%%
int main(void) {
    FILE *myfile = fopen("input.c", "r");
    if (!myfile) {
        printf("I can't open input.c!\n");
        return -1;
    }
    yyin = myfile;
    yyparse();
    printf("numbers = %d\nKeywords = %d\nIdentifiers = %d\nOperators = %d\n", dig, key, id, op);
    fclose(myfile);
    return 0;
}
void yyerror() {
    printf("EEK, parse error! Message");
}

```